

TalentDash - Engineering & Cost Optimization Architecture

Product Context

TalentDash is a:

- global company discovery platform
- salary insights platform
- interview experience platform
- workplace review platform
- career intelligence/search platform

Comparable products:

- Glassdoor
- AmbitionBox
- Levels.fyi

Primary audience:

- job seekers
- software engineers
- professionals
- students
- recruiters

Primary regions:

- US
- Europe
- India
- global English-speaking market

Primary Business Goals

Build a:

- global SEO-first internet platform
- ultra-low infra cost business
- acquisition-ready architecture
- scalable content/search platform
- highly cacheable web product
- long-runway bootstrap company

This is NOT:

- a real-time collaboration product
- a chat application
- a social network
- a heavy SaaS dashboard

This IS:

- a discovery platform
 - a structured data platform
 - a search/indexing business
 - a content distribution engine
-

Core Engineering Philosophy

Every engineering decision must optimize for:

1. Static-first architecture
 2. Maximum CDN delivery
 3. Minimal runtime compute
 4. Minimal database reads
 5. Global edge caching
 6. Predictable billing
 7. SEO scalability
 8. Fast mobile/web performance
 9. Small engineering team efficiency
 10. Acquisition-ready maintainability
-

MOST IMPORTANT RULE

NEVER COMPUTE PAGES AT REQUEST TIME IF THEY CAN BE PREGENERATED.

Goal:

User → CDN → Static HTML → Done

NOT:

User → Server → DB → SSR → Response

This single principle drastically reduces:

- infra cost
 - scaling risk
 - operational complexity
-

FINAL TECH STACK

Frontend

Framework

- Next.js 15 (App Router)

Styling

- Tailwind CSS

Rendering

- Static Site Generation (SSG)
- Incremental Static Regeneration (ISR)

React Strategy

- React Server Components
- Minimal client hydration

Avoid

- heavy SPA architecture
 - excessive client-side rendering
 - unnecessary runtime React logic
-

Hosting & CDN

Hosting

- Cloudflare Pages

CDN

- Cloudflare CDN

Why

- extremely cheap bandwidth
 - global edge delivery
 - excellent US/EU performance
 - predictable pricing
 - ideal for SEO-heavy products
-

Database

Primary Database

- Neon PostgreSQL

ORM

- Prisma ORM

Why PostgreSQL

- structured relational data
 - acquisition-friendly
 - excellent search support
 - scalable schema design
 - mature ecosystem
-

IMPORTANT DATABASE RULE

Database should NOT serve most traffic directly.

Database should mainly:

- store structured company/job/salary data
 - power admin systems
 - generate static pages
 - support search indexing
 - support background jobs
-

Search Strategy

Phase 1

Use:

- PostgreSQL Full Text Search

Phase 2

Add:

- Typesense

ONLY when:

- autocomplete quality matters
- typo tolerance becomes critical
- scale demands it

Avoid Initially

- Elasticsearch
- OpenSearch

These increase:

- infra cost
 - ops complexity
 - maintenance burden
-

Storage Layer

Object Storage

- Cloudflare R2

Used For

- company logos
 - screenshots
 - uploaded assets
 - generated reports
 - sitemap files
 - structured JSON exports
-

Background Jobs & Pipelines

Compute

- Google Cloud Run

Queue

- BullMQ

Redis

- Upstash Redis

Responsibilities

- scraping
 - salary normalization
 - metadata enrichment
 - SEO regeneration
 - sitemap generation
 - AI enrichment
 - deduplication jobs
-

AI Layer

AI Providers

- OpenAI
- Claude
- Gemini

AI Usage

- salary summarization
- review summarization
- metadata generation
- SEO enhancement
- company summaries

- FAQ generation
-

IMPORTANT AI RULE

AI should run:

- asynchronously
- offline
- in scheduled jobs

NEVER:

- run expensive AI calls on every request
 - build real-time AI chat systems initially
 - use self-hosted GPU infrastructure
-

Authentication

Recommended

- Clerk
OR
- Auth.js

Use Cases

- admin login
 - contributor system
 - review moderation
 - lightweight user accounts
-

Mobile Strategy

Phase 1

Responsive web app + PWA

Phase 2

React Native / Expo app

IMPORTANT

SEO traffic should remain the primary acquisition engine.

Do NOT overinvest in native apps early.

PAGE GENERATION ENGINE

This becomes the core moat.

Create an internal:

“Structured SEO Page Generation System”

Flow:

PostgreSQL Data

→ Generate Static Pages

→ Push To CDN

→ Serve Globally

This is how the business scales cheaply.

PAGE TYPES

Company Pages

/companies/google

Salary Pages

/salary/software-engineer/google

Interview Experience Pages

/interviews/meta/software-engineer

Comparison Pages

/compare/google-vs-meta

Role Pages

/roles/frontend-engineer

Geography Pages

/salaries/software-engineer/san-francisco

PAGE RENDERING STRATEGY

Page Type	Strategy
-----------	----------

Homepage	ISR
Company Pages	Static
Salary Pages	Static
Interview Pages	Static
Role Pages	Static
Location Pages	Static
Comparison Pages	ISR
Search Results	Dynamic
Admin Panel	SSR
Authentication	API/SSR

COST REDUCTION RULES

1. Avoid SSR Wherever Possible

SSR increases:

- compute cost
- scaling complexity
- infra unpredictability

Use static generation.

2. Avoid DB Queries Per Request

Goal:

- most traffic should never hit DB
-

3. Cache EVERYTHING

Cache:

- HTML
 - APIs
 - JSON
 - images
 - search payloads
 - salary datasets
 - sitemap files
-

4. Avoid Heavy Personalization Initially

Delay:

- realtime feeds
 - live notifications
 - chat
 - realtime collaboration
-

5. Avoid Heavy Frontend JS

Goal:

- fast rendering globally
 - excellent Core Web Vitals
 - low bandwidth usage
-

6. Avoid User Upload Complexity Initially

User uploads create:

- moderation burden
- storage cost
- abuse risk

Keep uploads minimal initially.

7. Avoid Kubernetes & Microservices

Use:

- clean monolith architecture

Do NOT overengineer.

SEO STRATEGY

The moat is:

- indexed pages
- long-tail SEO
- structured salary data
- internal linking
- freshness
- programmatic SEO
- crawl efficiency

NOT:

- realtime systems
 - fancy infra
 - AI hype
-

TEAM ENGINEERING PRIORITIES

Focus engineering effort on:

- structured data systems
- SEO page generation
- internal linking
- search indexing

- crawl optimization
- schema markup
- content freshness
- deduplication
- data quality

NOT:

- distributed systems
 - complex infra
 - overengineering
-

PERFORMANCE TARGETS

Metric	Target
LCP	< 2s
CLS	< 0.1
INP	< 200ms

ACQUISITION READINESS

The stack should look:

- modern
- scalable
- maintainable
- operationally clean
- professionally engineered

Potential acquirers should see:

- clean PostgreSQL schema
- scalable SEO architecture
- structured data moat
- predictable infra cost
- maintainable codebase

- low operational complexity

NOT:

- hobby project infra
- fragmented systems
- infra chaos
- overengineered architecture

ESTIMATED OPTIMIZED INFRA COST

Scale	Expected Monthly Cost
10k MAU	₹2k–₹5k
100k MAU	₹10k–₹25k
1M MAU	₹30k–₹80k

Possible ONLY if:

- pages remain mostly static
- CDN serves most traffic
- runtime compute stays minimal

FINAL ENGINEERING PRINCIPLE

Build:

“Global Static Internet Platform”

NOT:

“Heavy Dynamic Web Application”

This is the correct architecture for:

- SEO businesses
- salary intelligence products
- review aggregation platforms
- acquisition-focused startups
- bootstrap internet businesses